

Unit - V Regression and Classification

5.1 Linear Regression

Linear Regression is a supervised learning algorithm used to predict a continuous numerical value by modeling the relationship between independent variables (features) and a dependent variable (target).

The basic linear regression equation is:

$$y = \beta_0 + \beta_1 x + \varepsilon$$

Where:

- y = predicted output
- x = input feature
- β_0 = intercept
- β_1 = slope (coefficient)
- ε = error term

Assessing Performance of Regression / Metrics for Regression

The performance of a regression model is evaluated using error measures that compare actual values with predicted values.

01. Mean Absolute Error (MAE)

Measures the average absolute difference between actual and predicted values.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

02. Mean Squared Error (MSE)

Calculates the average squared difference between actual and predicted values.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

03. Root Mean Squared Error (RMSE)

Square root of MSE.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

04. Coefficient of Determination (R^2)

Represents the proportion of variance in the dependent variable explained by the model.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Where:

- SS_{res} = Residual Sum of Squares
- SS_{tot} = Total Sum of Squares

Interpretation

- ($R^2 = 1$): Perfect fit
- ($R^2 = 0$): Model explains no variance
- Negative value: Worse than predicting the mean

Short Examples

Consider the following actual and predicted values:

Observation	Actual (y)	Predicted ($\hat{y}$)
1	10	12
2	20	18
3	30	33

Errors ($y - \hat{y}$):

Observation	Error
1	-2
2	2
3	-3

01. Mean Absolute Error (MAE)

Formula:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

(The vertical bars $| |$ mean take the absolute value)

Calculation:

$$MAE = \frac{|-2| + |2| + |-3|}{3}$$

$$MAE = \frac{2 + 2 + 3}{3}$$

$$MAE = 2.33$$

Interpretation: On average, predictions are off by **2.33 units**.

Means suppose house price is 10 lakh and predicted 12 lakh then its off by 2.33 lakh.

02. Mean Squared Error (MSE)

Formula:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

The parentheses $()$ simply group the error, and the exponent ² squares it.

Calculation:

$$MSE = \frac{(-2)^2 + (2)^2 + (-3)^2}{3}$$

$$MSE = \frac{4 + 4 + 9}{3}$$

$$MSE = \frac{17}{3} = 5.67$$

Interpretation: The average squared error is **5.67**.

Because the errors are squared, the unit becomes: $(lakh\ rupee)^2$ which is not very intuitive.

Lower MSE = better model.

03. Root Mean Squared Error (RMSE)

Formula:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Calculation:

$$RMSE = \sqrt{5.67}$$

$$RMSE = 2.38$$

Interpretation: Predictions differ from actual values by about **2.38 units** on average.

By how much does the model typically miss the actual house price?" -

Approximately ₹2.38 lakh.

04. Coefficient of Determination (R^2)

Formula:

$$R^2 = 1 - \frac{SS_{res}}{SS_{res}}$$

Step 1: Calculate Mean of Actual Values

Formula:

$$\bar{y} = \frac{\sum y}{n}$$

Calculation:

$$\bar{y} = \frac{10+20+30}{3}$$

$$\bar{y} = \frac{60}{3}$$

$$\bar{y} = 20$$

Step 2: Calculate (SS_{res})

Formula:

$$SS_{res} = \sum (y_i - \hat{y}_i)^2$$

Calculation:

$$SS_{res} = (10 - 12)^2 + (20 - 18)^2 + (30 - 33)^2$$

$$SS_{res} = (-2)^2 + (2)^2 + (-3)^2$$

$$SS_{res} = 4 + 4 + 9$$

$$SS_{res} = 17$$

Step 3: Calculate (SS_{tot})

Formula:

$$SS_{tot} = \sum (y_i - \bar{y})^2$$

Calculation:

$$SS_{tot} = (10 - 20)^2 + (20 - 20)^2 + (30 - 20)^2$$

$$SS_{res} = (-10)^2 + (0)^2 + (10)^2$$

$$SS_{res} = 100 + 0 + 100$$

$$SS_{res} = 200$$

Step 4: Calculate R^2

Formula:

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Substitute values:

$$R^2 = 1 - \frac{17}{200}$$

$$R^2 = 1 - 0.085$$

$$R^2 = 0.915$$

Convert to percentage:

$$R^2 = 91.5\%$$

Interpretation: The model explains **91.5%** of the variation in the data, which indicates a very good fit.

So, MAE/RMSE tell you "how wrong" the predictions are, while R^2 tells you how well the model captures the overall pattern in the data.

Final Results

Metric	Value
<i>MAE</i>	2.33
<i>MSE</i>	5.67
<i>RMSE</i>	2.38
R^2	0.915 (91.5%)

Overfitting

Overfitting occurs when a model learns not only the underlying pattern but also the noise in the training data.

Characteristics

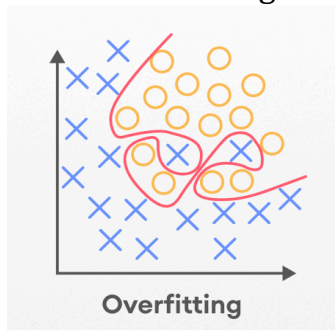
- A. Very low training error.
- B. High testing/validation error.
- C. Poor generalization to unseen data.

Example

Suppose a regression model is trained on house-price data.

- A. Training Accuracy: 99%
- B. Testing Accuracy: 65%

The model has memorized training data rather than learning the actual relationship.



The curve passes through almost every point, including noise.

Underfitting

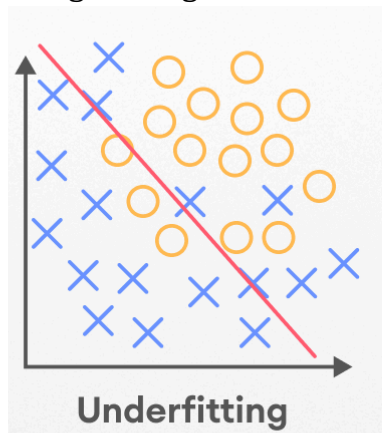
Underfitting occurs when a model is too simple to capture the underlying relationship in the data.

Characteristics

- A. High training error.
- B. High testing error.
- C. Poor performance on both seen and unseen data.

Example

Using a straight line to model a highly nonlinear relationship.



The model fails to capture the trend.

Catalysts (Causes) of Overfitting

Several factors can cause a regression model to overfit:

01. Excessive Model Complexity

- A. Too many features.
- B. High-degree polynomial regression.
- C. Large number of parameters.

Example:

Chatbot too complex → memorizes chats, not patterns.

02. Small Training Dataset

When the dataset is small, the model may memorize data points instead of learning patterns.

Example:

- 50 features with only 30 observations.

03. Presence of Noise

Noisy or incorrect data can mislead the model.

Examples:

- Typographical errors.
- Sensor measurement errors.
- Outliers.

04. Too Many Features (High Dimensionality)

Irrelevant variables increase model complexity and make fitting noise easier.

Example:

- Predicting house prices using 100 features when only 10 are useful.

05. Excessive Training

Training for too long may allow the model to learn noise along with useful patterns.

Example:

Overtrained chatbot learns noise, performs worse on new inputs.

5.2 Multiple Linear Regression:

Multiple Linear Regression is a statistical method used to predict a dependent variable using two or more independent variables.

Multiple Linear regression equation

$$Y = b_0 + b_1X_1 + b_2X_2 + b_3X_3 + \dots + b_nX_n$$

Where:

- **Y** = Dependent variable (target)
- **X₁, X₂, ... X_n** = Independent variables (features)
- **b₀** = Intercept
- **b₁, b₂, ... b_n** = Coefficients (weights)

Each feature contributes to the final prediction.

The model learns best values of coefficients to reduce error.

Implementation of multiple linear regression

1. Collect dataset
2. Select features (X) and target (Y)
3. Split data into training and testing sets
4. Train the model using training data
5. Predict using test data
6. Evaluate performance (e.g., MSE, R^2 score)

5.3 Metrics for Regression: Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE)

Referer page no. 01 Assessing Performance of Regression

5.4 Logistic Regression

Logistic Regression is a **classification algorithm** used when the output is **categorical** (not continuous), such as:

- Yes / No
- Spam / Not spam
- Disease / No disease

It predicts **probability**, not direct values.

Binary Classification

Binary classification means **only two classes exist**.

Examples

- Email: Spam or Not Spam
- Loan: Approved or Rejected
- Disease: Positive or Negative

Decision Rule

- If $P \geq 0.5 \rightarrow \text{Class 1}$
- If $P < 0.5 \rightarrow \text{Class 0}$

Multiclass Classification, Handling more than two classes

Multiclass classification means **more than two classes exist**.

Examples

- Animal: Cat / Dog / Horse
- Handwritten digits: 0–9
- Fruit: Apple / Banana / Orange

Logistic Regression is naturally binary, so we extend it using strategies:

One-vs-One

Train a classifier for **every pair of classes**.

Example (3 classes: A, B, C)

We train:

- A vs B
- A vs C
- B vs C

Prediction

Each classifier votes:

- Winner gets 1 vote
- Final class = most votes

One-vs-Rest

For **K classes**, train **K separate binary classifiers**.

Each classifier answers:

“Is this class OR not this class?”

Example (3 classes: A, B, C)

We build:

1. Classifier A vs (B + C)
2. Classifier B vs (A + C)
3. Classifier C vs (A + B)

5.5 Metrics for Classification / Assessing Classification Performance

Confusion Matrix

A table that shows correct and incorrect predictions of a classification model.

Representation (Spam Example)

	Predicted Spam	Predicted Not Spam
Actual Spam	TP	FN
Actual Not Spam	FP	TN

Meaning

- A. TP → True Positive
- B. FP → False Positive
- C. FN → False Positive
- D. TN → True Negative

Example:

1 = Spam, 0 = Not Spam

Meaning of Representation:

- A. TP → Spam correctly detected
- B. FP → Not spam marked as spam
- C. FN → Spam missed
- D. TN → Correctly rejected not spam

Calculation:

	Predicted Spam	Predicted Not Spam
Actual Spam	TP = 2	FN = 1
Actual Not Spam	FP = 1	TN = 1

Accuracy

Proportion of correctly classified emails.

Formula:

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

Calculation:

$$Accuracy = \frac{2+1}{2+1+1+1}$$

$$Accuracy = \frac{3}{5} = 0.6$$

$$Accuracy = 60\%$$

Precision

Out of predicted spam emails, how many are actually spam.

Formula:

$$Precision = \frac{TP}{TP+FP}$$

Calculation:

$$Precision = \frac{2}{2+1}$$

$$Precision = \frac{2}{3} = 0.667$$

$$Precision = 66.7\%$$

Recall

Out of actual spam emails, how many were correctly detected.

Formula:

$$Recall = \frac{TP}{TP+FN}$$

Calculation:

$$Recall = \frac{2}{2+1}$$

$$Recall = \frac{2}{3} = 0.667$$

$$Recall = 66.7\%$$

F1 Score

Balanced measure of precision and recall.

Formula:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Calculation:

$$F1 = 2 \cdot \frac{0.667 \cdot 0.667}{0.667 + 0.667}$$

$$F1 = 0.667$$

$$F1 = 66.7\%$$

ROC (Receiver Operating Characteristic) Curve

Trade-off between **True Positive Rate (TPR)** and **False Positive Rate (FPR)** at different thresholds.

Formula

$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

Calculation:

$$TPR = \frac{2}{3} = 0.667$$

$$FPR = \frac{1}{2} = 0.5$$

$$ROC \text{ point} = (0.5, 0.667)$$

Spam Meaning

- TPR → how many spam caught
- FPR → how many normal emails wrongly marked spam

AUC (Area Under Curve)

How well the model separates Spam vs Not Spam.

Representation

- 1 → perfect spam detector
- 0.5 → random guessing
- < 0.5 → poor model

Example:

Email	Actual	Score
-------	--------	-------

A	1 (Spam)	0.9
B	0 (Not Spam)	0.8
C	1 (Spam)	0.7
D	0 (Not Spam)	0.6

01. Separate Positive and Negative

Positives (Spam = 1): A → 0.9, C → 0.7

Negatives (Not Spam = 0): B → 0.8, D → 0.6

AUC = Probability that a **random positive score > random negative score**

02. Make all comparisons

We compare each **positive vs negative**

Total comparisons = $2 \times 2 = 4$

03. Compare:

A (0.9)

- vs B (0.8) → ✓ (0.9 > 0.8)
- vs D (0.6) → ✓ (0.9 > 0.6)

Wins = 2

C (0.7)

- vs B (0.8) → ✗ (0.7 < 0.8)
- vs D (0.6) → ✓ (0.7 > 0.6)

Wins = 1

04. Total wins

Correct rankings = $2 + 1 = 3$, Total comparisons = 4

05. AUC Formula

$$AUC = \frac{\text{Correct rankings}}{\text{Total comparisons}}$$

$$AUC = \frac{3}{4} = 0.75$$

AUC = 0.75, meaning the classifier has a good ability to distinguish spam from non-spam emails.